Optimizers: SGD, Mini-batch SGD ve Momentum

Atıl Samancıoğlu

1 Optimizer Nedir?

Bir sinir ağında optimizer'ın tek bir temel görevi vardır:

> *Loss fonksiyonunu minimize edecek şekilde ağırlıkları güncellemek*

Forward propagation sonunda bir tahmin ($\hat{y}$) elde ederiz. Loss hesaplandıktan sonra optimizer devreye girer ve **backpropagation** sırasında ağırlıkları günceller.

Temel güncelleme fikri her optimizer için aynıdır:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

Burada:

- η: learning rate
- $\frac{\partial L}{\partial w}$: loss'un ağırlığa göre türevi

Farklı optimizer'lar bu güncellemeyi **nasıl yaptıkları** açısından ayrılır.

—

2 Stochastic Gradient Descent (SGD)

Gradient Descent'te tüm veri seti kullanılırken, **SGD** bu yaklaşımı radikal biçimde değiştirir.

2.1 Temel Fikir

> *Her iteration'da sadece **tek bir veri noktası** kullanılır*

Yani:

- Batch size = 1
- Her veri noktası → ayrı bir ağırlık güncellemesi

—

2.2 Epoch ve Iteration Davranışı

Örnek:

- Veri seti: 1000 örnek
- Batch size: 1

Bu durumda:

$$1 \text{ epoch } = 1000 \text{ iteration}$$

Her iteration'da:

1. Tek bir veri alınır
2. Forward propagation yapılır
3. Loss hesaplanır
4. Ağırlıklar güncellenir

—

2.3 Avantajları

- Çok düşük bellek ihtiyacı
- Düşük donanımlı sistemlerde çalışabilir
- Online learning için uygundur

—

2.4 Dezavantajları

SGD'nin en büyük problemi **gürültüdür (noise)**.

Tek veriyle güncelleme yapıldığı için:

- Gradient yönü sürekli değişir
- Loss zig-zag yaparak ilerler
- Convergence yavaş ve dengesiz olur

Sezgisel olarak:

Model her adımda farklı bir yöne çekilir

—

3 Mini-batch SGD

Mini-batch SGD, SGD'nin gürültü problemini azaltmak için geliştirilmiştir.

3.1 Temel Fikir

> *Her iteration'da tek veri yerine küçük bir veri grubu kullanılır*

Bu gruba **batch** denir.

—

3.2 Örnek Senaryo

- Veri seti: 100.000 örnek
- Batch size: 1000

Bu durumda:

$$1 \text{ epoch } = \frac{100000}{1000} = 100 \text{ iteration}$$

Her iteration'da:

- 1000 veri birlikte işlenir
- Ortalama gradient hesaplanır
- Ağırlıklar bir kez güncellenir

—

3.3 Neden Daha İyi?

Mini-batch kullanıldığında:

- Gürültü azalır
- Gradient daha stabil olur
- GPU kullanımı daha verimli hale gelir

Sezgisel olarak:

> *Tek kişinin fikri yerine küçük bir grubun ortalaması alınır*

—

3.4 Avantajları

- SGD'ye göre daha hızlı convergence
- Daha az noise
- Donanım kaynakları daha verimli kullanılır

—

3.5 Dezavantajı

- Noise tamamen ortadan kalkmaz
- Hâlâ zig-zag davranış görülebilir

Bu bizi bir sonraki adıma götürür: **Momentum**.

—

4 SGD with Momentum

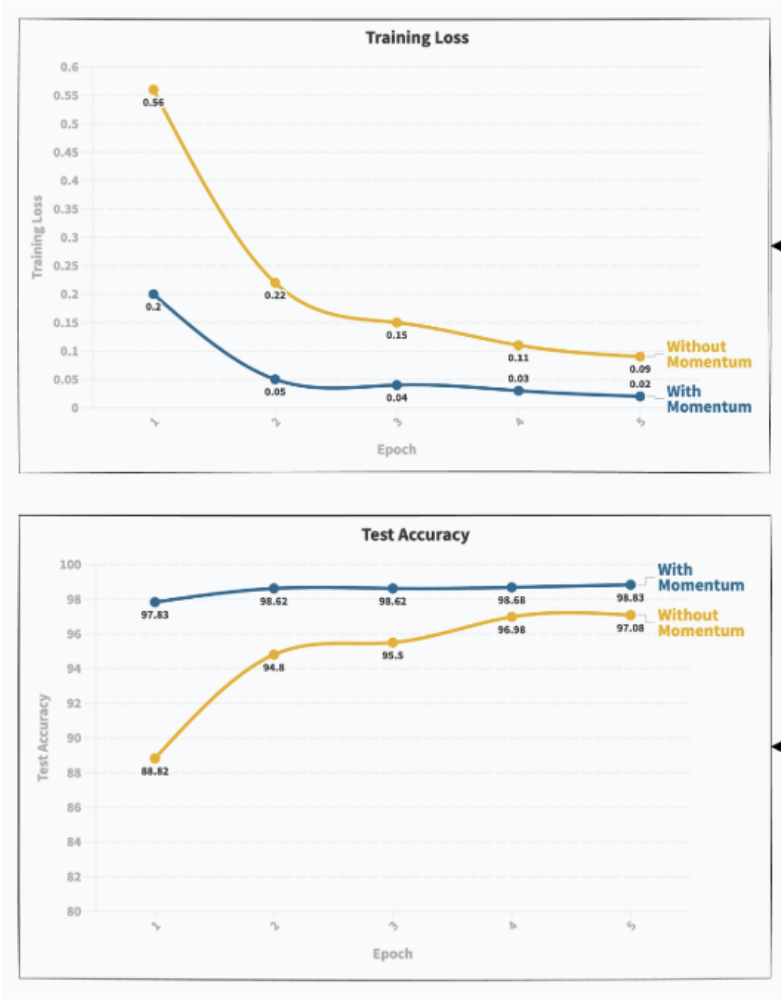

Figure 1: Momentum SGD vs SGD

Momentum'un amacı çok nettir:

Gradient yönündeki gürültüyü yumuşatmak

—

4.1 Sezgisel Anlatım

Bir topu yokuş aşağı yuvarladığınızı düşünün:

- Küçük taşlar yönü biraz değiştirir
- Ama topun **momentumu** varsa yönünü kolay kolay kaybetmez

Momentum SGD de aynısını yapar.

—

4.2 Exponential Weighted Average

Momentum, geçmiş gradient'lerin etkisini tamamen silmez. Onları **azalarak** taşır.

$$v_t = \beta v_{t-1} + (1 - \beta)\frac{\partial L}{\partial w_t}$$

$$w_t = w_{t-1} - \eta v_t$$

Burada:

- v_t: momentum terimi
- β: momentum katsayısı (genelde 0.9)

—

4.3 Ne İşe Yarar?

- Gürültüyü yumuşatır
- Zig-zag hareketleri azaltır
- Daha hızlı ve kararlı convergence sağlar

Sezgisel olarak:

Model sadece bugünkü gradient'e değil, geçmiş yönlere de bakar

—

5 Özet Karşılaştırma

Optimizer	Batch Size	Noise	Convergence
SGD	1	Çok Yüksek	Yavaş
Mini-batch SGD	32–1024	Orta	Daha Hızlı
Momentum SGD	32–1024	Düşük	Hızlı

—

6 Bir Sonraki Adım

Bir sonraki bölümde:

- Adagrad
- RMSProp
- Adam Optimizer
- Dynamic learning rate

incelenecektir.